```
!pip install ultralytics opencv-python matplotlib -q

print("Libraries installed successfully!")
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 46.7/46.7 kB 1.2 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.5/1.5 MB 10.5 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 53.2/53.2 kB 2.8 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 87.5/87.5 kB 1.9 MB/s eta 0:00:00
Libraries installed successfully!
```

```
from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt
import numpy as np

print("Libraries imported successfully!")
```

```
Creating new Ultralytics Settings v0.0.8 file ✅
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics
Libraries imported successfully!
```

```
# Load pre-trained YOLOv8 model
model = YOLO("yolov8n.pt")

print("YOLOv8 model loaded successfully!")
```

```
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt to 'yolov8n.pt': 100% ━━━━━━━━━━━━ 6.2MB 2
YOLOv8 model loaded successfully!
```

```
!wget -q https://ultralytics.com/images/bus.jpg -O bus.jpg

print("Image downloaded successfully!")
```

```
Image downloaded successfully!
```

```
image = cv2.imread("bus.jpg")

# Convert BGR to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(10, 7))
plt.imshow(image_rgb)
plt.axis("off")
plt.title("Original Image")
plt.show()
```




```
results = model("bus.jpg")

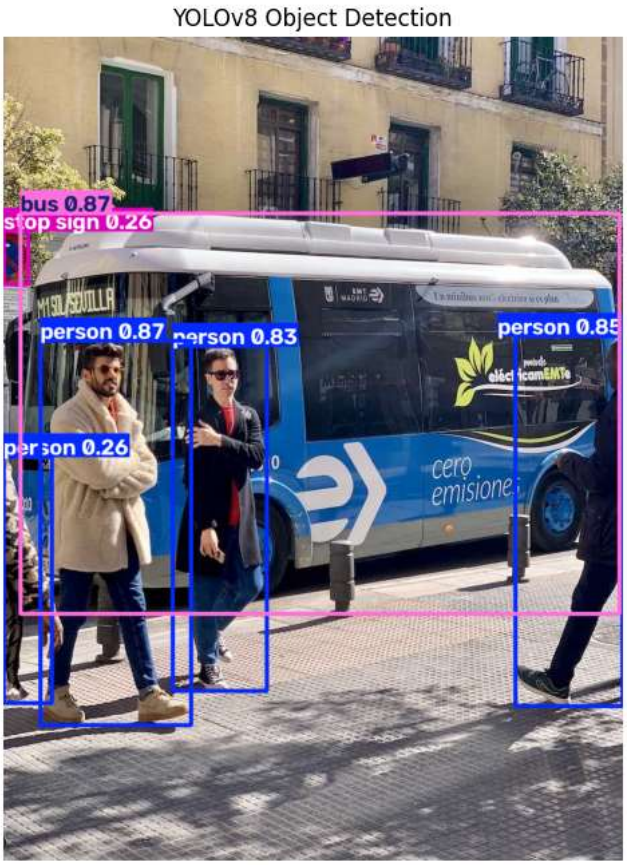
print("Object detection completed!")
```

```
image 1/1 /content/bus.jpg: 640x480 4 persons, 1 bus, 1 stop sign, 473.3ms
Speed: 26.3ms preprocess, 473.3ms inference, 32.1ms postprocess per image at shape (1, 3, 640, 480)
Object detection completed!
```

```
result_image = results[0].plot()

result_image_rgb = cv2.cvtColor(
    result_image,
    cv2.COLOR_BGR2RGB
)

plt.figure(figsize=(12, 8))
plt.imshow(result_image_rgb)
plt.axis("off")
plt.title("YOLOv8 Object Detection")
plt.show()
```




```
result = results[0]

print("Detected Objects:")
print("-" * 40)

for box in result.boxes:

    class_id = int(box.cls[0])
    confidence = float(box.conf[0])

    class_name = model.names[class_id]

    print(
        f"Object: {class_name:15s} "
        f"Confidence: {confidence:.2f}"
    )
```

```
Detected Objects:
----------------------------------------
Object: bus             Confidence: 0.87
Object: person          Confidence: 0.87
Object: person          Confidence: 0.85
Object: person          Confidence: 0.83
Object: person          Confidence: 0.26
Object: stop sign       Confidence: 0.26
```

```
print("Bounding Box Information")
print("-" * 60)

for box in result.boxes:

    class_id = int(box.cls[0])
    class_name = model.names[class_id]
    confidence = float(box.conf[0])

    x1, y1, x2, y2 = box.xyxy[0].tolist()

    print(f"Object      : {class_name}")
    print(f"Confidence  : {confidence:.2f}")
    print(
```

```
        f"Bounding Box: "
        f"({x1:.1f}, {y1:.1f}, {x2:.1f}, {y2:.1f})"
    )
    print("-" * 60)
```

```
Bounding Box Information
------------------------------------------------------------
Object      : bus
Confidence  : 0.87
Bounding Box: (22.9, 231.3, 805.0, 756.8)
------------------------------------------------------------
Object      : person
Confidence  : 0.87
Bounding Box: (48.6, 398.6, 245.3, 902.7)
------------------------------------------------------------
Object      : person
Confidence  : 0.85
Bounding Box: (669.5, 392.2, 809.7, 877.0)
------------------------------------------------------------
Object      : person
Confidence  : 0.83
Bounding Box: (221.5, 405.8, 345.0, 857.5)
------------------------------------------------------------
Object      : person
Confidence  : 0.26
Bounding Box: (0.0, 550.5, 63.0, 873.4)
------------------------------------------------------------
Object      : stop sign
Confidence  : 0.26
Bounding Box: (0.1, 254.5, 32.6, 324.9)
------------------------------------------------------------
```

```
object_counts = {}

for box in result.boxes:

    class_id = int(box.cls[0])
    class_name = model.names[class_id]

    if class_name in object_counts:
        object_counts[class_name] += 1
    else:
        object_counts[class_name] = 1

print("Object Count")
print("-" * 30)

for object_name, count in object_counts.items():
    print(f"{object_name}: {count}")
```

```
Object Count
------------------------------
bus: 1
person: 4
stop sign: 1
```

```
confidences = [
    float(box.conf[0])
    for box in result.boxes
]

if len(confidences) > 0:

    average_confidence = np.mean(confidences)
    highest_confidence = np.max(confidences)
    lowest_confidence = np.min(confidences)

    print("Confidence Score Analysis")
    print("-" * 35)
    print(f"Average Confidence : {average_confidence:.3f}")
    print(f"Highest Confidence : {highest_confidence:.3f}")
    print(f"Lowest Confidence  : {lowest_confidence:.3f}")

else:
    print("No objects detected.")
```

```
Confidence Score Analysis
-----------------------------------
Average Confidence : 0.656
```

```
Highest Confidence : 0.873
Lowest Confidence  : 0.255
```

```
def calculate_iou(box1, box2):

    x1 = max(box1[0], box2[0])
    y1 = max(box1[1], box2[1])

    x2 = min(box1[2], box2[2])
    y2 = min(box1[3], box2[3])

    intersection_width = max(0, x2 - x1)
    intersection_height = max(0, y2 - y1)

    intersection_area = (
        intersection_width * intersection_height
    )

    area1 = (
        (box1[2] - box1[0]) *
        (box1[3] - box1[1])
    )

    area2 = (
        (box2[2] - box2[0]) *
        (box2[3] - box2[1])
    )

    union_area = area1 + area2 - intersection_area

    if union_area == 0:
        return 0

    return intersection_area / union_area


print("IoU function created successfully!")
```

```
IoU function created successfully!
```

```
import time

start_time = time.time()

results_speed = model("bus.jpg")

end_time = time.time()

inference_time = end_time - start_time

print(f"Inference Time: {inference_time:.4f} seconds")
print(
    f"Approximate FPS: "
    f"{1 / inference_time:.2f}"
)
```

```

image 1/1 /content/bus.jpg: 640x480 4 persons, 1 bus, 1 stop sign, 250.4ms
Speed: 8.3ms preprocess, 250.4ms inference, 1.7ms postprocess per image at shape (1, 3, 640, 480)
Inference Time: 0.2805 seconds
Approximate FPS: 3.57
```

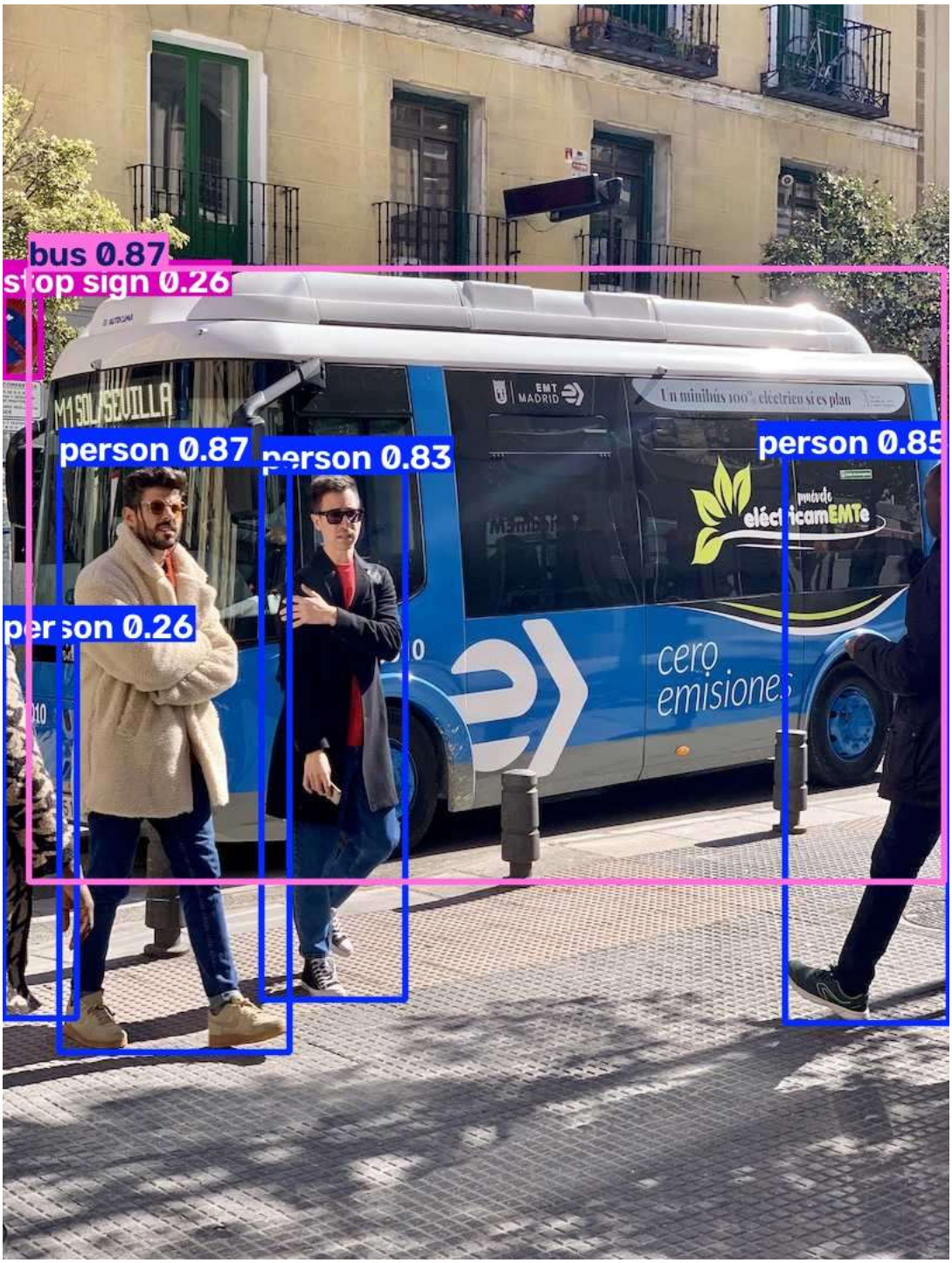
```
results[0].save(
    filename="detected_objects.jpg"
)

print("Detection result saved as detected_objects.jpg")
```

```
Detection result saved as detected_objects.jpg
```

```
from IPython.display import Image, display

display(Image(filename="detected_objects.jpg"))
```



OBSERVATIONS

1. YOLOv8 successfully detected multiple objects present in the input image.
2. Bounding boxes were generated around the detected objects.
3. Each detected object was assigned a class label.
4. A confidence score was provided for every detection.
5. Objects with higher confidence scores were detected more reliably.
6. Detection performance can be affected by image quality, object size, illumination, occlusion, and background complexity.
7. The pre-trained model can perform object detection without training a new model from scratch.
8. Inference speed depends on the model size and available computational hardware.

MODEL COMPARISON

YOLO:

- Single-stage object detector.
- Performs detection and localization in a single network.
- Designed for fast inference.
- Suitable for real-time applications.

SSD:

- Single-stage object detector.
- Uses multiple feature maps for detecting objects at different scales.
- Provides a balance between speed and detection accuracy.
- Suitable for real-time applications.

Faster R-CNN:

- Two-stage object detection approach.
- Uses a Region Proposal Network followed by object classification.
- Generally provides strong detection accuracy.
- Usually has higher computational requirements than lightweight single-stage detectors.

Theory Questions

Q1. Differentiate between image classification, object detection, and image segmentation.

Answer:

Image Classification assigns one or more class labels to an entire image. It does not identify the exact location of objects.

Object Detection identifies objects in an image and determines their locations using bounding boxes along with class labels.

Image Segmentation divides an image into different regions or pixels and assigns a class to each region, providing detailed information about object boundaries.

Feature	Image Classification	Object Detection	Image Segmentation
Output	Class label	Bounding box + class label	Pixel-level regions
Object location	Not provided	Provided	Precisely provided
Detail	Low	Medium	High
Example	"Dog"	Dog + bounding box	Pixels belonging to dog

Q2. Explain the working principle of the YOLO (You Only Look Once) object detection algorithm.

Answer:

YOLO stands for You Only Look Once. It is a deep learning-based object detection algorithm that performs object detection in a single network.

The input image is passed through the neural network, which predicts the bounding boxes, object classes, and confidence scores. The predictions are then processed to remove unsuitable overlapping detections.

Since the complete image is processed in a single inference process, YOLO provides fast object detection and is suitable for real-time applications.

Q3. Compare YOLO, SSD, and Faster R-CNN based on speed, accuracy, and practical applications.

Answer:

Parameter	YOLO	SSD	Faster R-CNN
Architecture	Single-stage	Single-stage	Two-stage
Speed	High	High	Generally lower
Accuracy	High	Good	Generally strong
Real-time use	Suitable	Suitable	More computationally demanding
Main use	Real-time detection	Real-time detection	High-quality detection
Processing	Single detection network	Multiple feature maps	Region proposals + classification

YOLO and SSD are generally preferred when fast detection is required, while Faster R-CNN uses a two-stage approach and is more computationally demanding.

Q4. What is a bounding box? Why is it important in object detection?

Answer:

A bounding box is a rectangular box drawn around an object detected in an image.

It is generally represented using coordinates such as:

(x1, y1, x2, y2)

A bounding box is important because it indicates the location and approximate size of an object in an image. It helps the object detection system identify which region of the image contains the detected object.

Q5. Explain the significance of confidence score and Intersection over Union (IoU) in object detection.

Answer:

A confidence score represents how strongly the detection model believes that a particular detected object belongs to the predicted class.

For example:

Person → 0.92

This means that the model assigned a confidence value of 0.92 to the detection.

Intersection over Union (IoU) measures the overlap between two bounding boxes.

IoU = Area of Intersection / Area of Union

IoU ranges from 0 to 1.

0 means there is no overlap, while 1 represents perfect overlap.

IoU is commonly used to evaluate how closely a predicted bounding box matches a reference bounding box.

Q6. What are the advantages of using pre-trained deep learning models over traditional image processing techniques?

Answer:

The advantages of using pre-trained deep learning models include:

1. They can detect complex objects automatically.
2. They can learn high-level visual features.
3. They can recognize multiple object categories.
4. They reduce the need to manually design image-processing features.
5. They can be adapted to different applications using transfer learning.
6. They can provide object localization as well as classification.
7. They are suitable for complex real-world environments.

Q7. Why are datasets such as MS COCO and Pascal VOC widely used for object detection research?

Answer:

MS COCO and Pascal VOC are widely used because they provide standardized datasets containing images with object annotations.

They allow researchers and students to:

1. Train object detection models.
2. Test detection performance.
3. Compare different algorithms.
4. Evaluate object localization.
5. Study detection across different object categories.

These datasets provide a common basis for evaluating object detection systems.

Q8. Mention five real-world applications of object detection in computer vision.

Answer:

Five real-world applications of object detection are:

1. Autonomous vehicles
2. Video surveillance
3. Healthcare
4. Smart city systems
5. Retail analytics

Other applications include industrial inspection and traffic monitoring.

Q9. What challenges are commonly encountered while detecting objects in complex real-world environments?

Answer:

Common challenges in object detection include:

1. Poor illumination
2. Object occlusion
3. Small objects